

Python for Web Development

Lesson 2: HTTP Basics and Flask Fundamentals

HTTP Basics and Flask Fundamentals – Study Notes

Key Concepts

1. **HTTP request/response cycle** – methods, status codes, headers, and bodies.
2. **Flask routing** – mapping URLs to view functions with `@app.route``.
3. **Template rendering** – using Jinja2 to generate dynamic HTML (`render_template``).
4. **Request handling** – accessing query strings, form data, JSON payloads via `request``.
5. **Application structure** – separating routes, templates, static files, and configuration for maintainable projects.

Summary

HTTP (Hypertext Transfer Protocol) is the foundation of web communication. A client sends an **HTTP request**—which includes a method (GET, POST, etc.), a URL, optional headers, and an optional body—to a server. The server processes the request and returns an **HTTP response** containing a status code (e.g., 200 OK, 404 Not Found), response headers, and a body (often HTML, JSON, or other media). Understanding the semantics of methods and status codes is essential for building correct web APIs and web pages.

Flask is a lightweight Python web framework that lets you define routes (URL patterns) and associate them with Python functions called **view functions**. When a request matches a route, Flask calls the corresponding view, which can read data from the request, perform business logic, and return a response. By default, Flask uses **Jinja2** for templating, allowing you to embed Python-like expressions inside HTML files. `render_template`` merges a template with a context dictionary and returns the rendered HTML as the response body. Together, these concepts let you create dynamic, data driven web applications with minimal boilerplate.

Important Points

- **HTTP Methods**
 - `GET``: retrieve data, safe & idempotent, parameters in query string.
 - `POST``: submit data, body contains form data or JSON, not idempotent.
 - `PUT`` / `PATCH``: update resources.
 - `DELETE``: remove resources.
- **Status Codes**
 - 2xx – success (200 OK, 201 Created).
 - 3xx – redirection (301 Moved Permanently, 302 Found).
 - 4xx – client error (400 Bad Request, 401 Unauthorized, 404 Not Found).
 - 5xx – server error (500 Internal Server Error).

- **Flask Routing**

```
python
```

```
@app.route('/path', methods=['GET', 'POST'])
```

```
def view_func():
```

```
...
```

```
...
```

- Variable rules: `<int:id>`, `<string:name>`, `<path:subpath>`.
- `endpoint` name defaults to the function name; can be overridden with `endpoint=`.

- **Request Object** (`flask.request`)

- `request.args` ! query parameters (`?page=2`).
- `request.form` ! form encoded POST data.
- `request.json` ! parsed JSON body (`application/json`).
- `request.files` ! uploaded files.
- `request.method` ! HTTP verb used.

- **Response Generation**

- Return a string, dict (auto JSON), or a `Response` object.
- Use `make\_response` to set cookies, custom headers, or status codes.

- **Template Rendering**

- Place HTML files in a `templates/` folder.
- Use Jinja2 syntax: `{% variable %}`, `{% for item in list %}...{% endfor %}`.
- Pass context: `render\_template('page.html', user=user, items=items)`.

- **Project Layout** (recommended)

```
...
```

```
myapp/
```

```
% % app.py          # Flask app factory / entry point
```

```
% % config.py       # configuration classes
```

```
% % requirements.txt
```

```
% % templates/
```

```
% % % base.html
```

```
% % static/
```

```
% % % css/
```

```
% % % js/
```

```
% % routes/
```

```
% % __init__.py     # blueprint registrations
```

```
...
```

- **Debugging**

- `app.run(debug=True)` enables the interactive debugger and auto reload.
- Use `flask shell` for an interactive context with the app loaded.

```
---
```

Examples

1p ã Simple “Hello, World!” with a template

****File structure****

myapp/

% % app.py

% % templates/

 % % hello.html

****app.py****

```python

from flask import Flask, render\_template, request

def create\_app():

    app = Flask(\_\_name\_\_)

    @app.route('/', methods=['GET'])

    def index():

        # Pull a name from the query string, default to "World"

        name = request.args.get('name', 'World')

        return render\_template('hello.html', name=name)

    return app

if \_\_name\_\_ == '\_\_main\_\_':

    create\_app().run(debug=True)

---

**\*\*templates/hello.html\*\***

```html

<!doctype html>

<html lang="en">

<head>

 <meta charset="utf-8">

 <title>Hello</title>

</head>

<body>

 <h1>Hello, {{ name }}!</h1>

 <p>Try / ?name=Flask</p>

</body>

</html>

***Explanation*:**

- The route `/'` handles only GET requests.
- `request.args.get('name')` reads the query string parameter.
- `render\_template` injects `name` into the Jinja2 template, producing a personalized page.

2b ã Form handling – POST !' JSON response

****File structure****

...

myapp/

% % app.py

% % templates/

% % contact.html

...

****app.py****

```python

from flask import Flask, render\_template, request, jsonify

def create\_app():

app = Flask(\_\_name\_\_)

@app.route('/contact', methods=['GET', 'POST'])

def contact():

if request.method == 'POST':

# Expect JSON: {"email": "...", "message": "..."}  
 data = request.get\_json()

email = data.get('email')

message = data.get('message')

# In a real app you'd store or email this data

return jsonify({

'status': 'success',

'email': email,

'msg\_len': len(message or '')

}), 201 # 201 Created

# GET !' show the HTML form

return render\_template('contact.html')

return app

if \_\_name\_\_ == '\_\_main\_\_':

create\_app().run(debug=True)

...

**\*\*templates/contact.html\*\***

```html

<!doctype html>

<html lang="en">

<head>

<meta charset="utf-8">

<title>Contact</title>

</head>

<body>

<h2>Send a Message</h2>

<form id="msgForm">

```

Email: <input type="email" name="email" required><br>
Message: <textarea name="message" required></textarea><br>
<button type="submit">Send</button>
</form>
<script>
const form = document.getElementById('msgForm');
form.addEventListener('submit', async e => {
  e.preventDefault();
  const payload = {
    email: form.email.value,
    message: form.message.value
  };
  const resp = await fetch('/contact', {
    method: 'POST',
    headers: {'Content-Type': 'application/json'},
    body: JSON.stringify(payload)
  });
  const result = await resp.json();
  alert(JSON.stringify(result));
});
</script>
</body>
</html>
...

```

Explanation:

- The same URL handles both GET (serve the form) and POST (process JSON).
- ``request.get_json()`` parses the incoming JSON body.
- ``jsonify`` builds a JSON response; we also set a 201 status code to indicate a new resource was created.

Quick Review

1. **What is the difference between ``request.args`` and ``request.form``?**
2. **Name two idempotent HTTP methods and explain why idempotence matters.**
3. **How do you pass data from a Flask view to a Jinja2 template?**
4. **Write a route that captures an integer ``post_id`` from the URL and returns a JSON object ``{ "post_id": post_id }``.**
5. **Which Flask function would you use to set a custom response header, and how?**

Further Reading

- **Flask Blueprints** – modularizing large applications.
- **Flask RESTful / Flask API** – building RESTful services with Flask.

- **HTTP/2 & HTTP/3** – performance improvements over HTTP/1.1.
- **Jinja2 Advanced Features** – filters, macros, inheritance, and custom tests.
- **Security Best Practices** – CSRF protection, input validation, and HTTPS enforcement.

Happy coding! ☺

